

1. JavaScript Basics

What is JavaScript?

JavaScript is a **dynamic, interpreted scripting language** used to build **interactive web applications**. It runs in the **browser** and also on servers (Node.js).

Core JavaScript Concepts

1. Variables

```
let name = "Angular";
```

```
const version = 17;
```

```
var oldWay = true;
```

- `let` → block scoped, recommended
 - `const` → cannot be reassigned
 - `var` → function scoped (avoid)
-

2. Data Types

- Primitive: string, number, boolean, null, undefined
- Reference: object, array, function

```
let age = 25;
```

```
let isActive = true;
```

```
let skills = ["JS", "TS"];
```

```
let user = { id: 1, name: "San" };
```

3. Functions

```
function add(a, b) {
```

```
  return a + b;
```

```
}
```

```
// Arrow function  
const multiply = (x, y) => x * y;
```

4. Control Statements

```
if (age > 18) {  
  console.log("Adult");  
}
```

```
for (let i = 0; i < 3; i++) {  
  console.log(i);  
}
```

5. Arrays & Objects

```
let products = ["Laptop", "Mobile"];  
products.push("Tablet");
```

```
let emp = {  
  id: 1,  
  name: "John"  
};
```

6. Asynchronous JavaScript

```
setTimeout(() => {  
  console.log("Executed later");  
}, 2000);
```

```
fetch("/api/data")  
  .then(res => res.json())  
  .then(data => console.log(data));
```

2. TypeScript Basics

What is TypeScript?

TypeScript is a **superset of JavaScript** that adds:

- **Static typing**
- **Compile-time error checking**
- **OOP features**

Angular is written **entirely in TypeScript**.

Why TypeScript for Angular?

- Detects errors early
 - Improves code readability
 - Strong support for large-scale applications
-

Core TypeScript Concepts

1. Typed Variables

```
let username: string = "Angular";
```

```
let count: number = 10;
```

```
let isValid: boolean = true;
```

2. Arrays & Tuples

```
let numbers: number[] = [1, 2, 3];
```

```
let userInfo: [number, string] = [1, "Admin"];
```

3. Functions with Types

```
function add(a: number, b: number): number {  
    return a + b;  
}
```

4. Interfaces (Very Important for Angular)

```
interface User {  
    id: number;  
    name: string;  
    email?: string; // optional  
}
```

```
let user: User = {  
    id: 1,  
    name: "San"  
};
```

5. Classes & Constructors

```
class Employee {  
    constructor(  
        public id: number,  
        public name: string  
    ) {}  
}
```

```
let emp = new Employee(1, "John");
```

6. Enums

```
enum Role {  
    Admin,  
    User,  
    Guest  
}
```

```
let myRole: Role = Role.Admin;
```

7. Access Modifiers

- public (default)
- private
- protected

```
class Account {  
    private balance: number = 0;  
}
```

3. JavaScript vs TypeScript (Quick Comparison)

Feature	JavaScript	TypeScript
Typing	Dynamic	Static
Compilation	Not required	Required
Error Detection	Runtime	Compile-time
Angular Usage	Limited	Mandatory

4. How This Connects to Angular

Angular uses:

- **TypeScript classes** → Components & Services
- **Interfaces** → API Models
- **Arrow functions** → Observables & Callbacks
- **Async concepts** → HTTP calls